PART A

Experiment 5

A.1 Aim: Building an Automated CI/CD Pipeline using Jenkins and GitHub Webhooks

A.2 Prerequisite.

Windows PC, Git installed, Jetkin , GitHub ,basic command prompt knowledge.

A.3 Outcome

Students can analyze the CI CD process.

A.4 Theory

What is CI/CD?

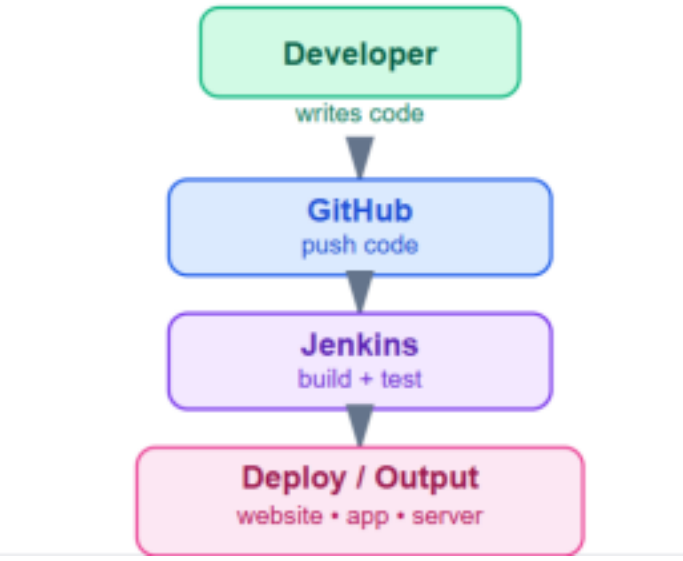
Imagine you're writing a Python project.

Without CI/CD:

1. You write code.
2. Save it on your computer.
3. Manually test it.
4. Manually upload it to GitHub.
5. Manually deploy it.

Every time you make a change, you repeat these steps.

With CI/CD, this becomes automatic.



CI = Continuous Integration → Automatically build and test whenever code changes.

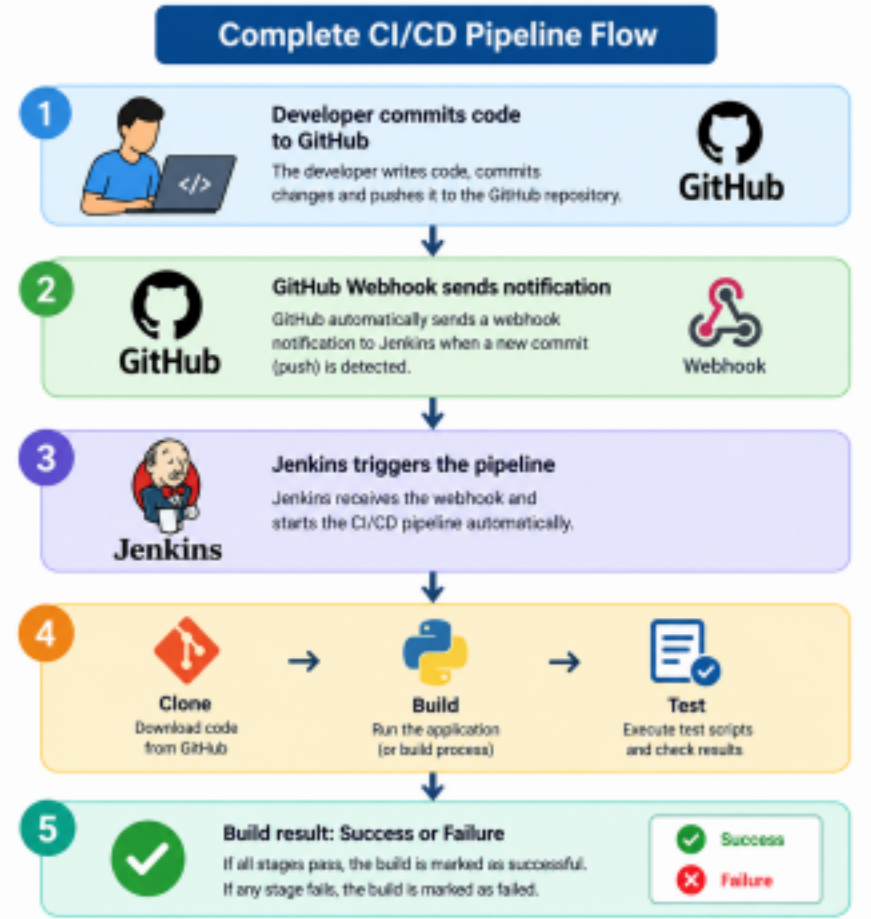
CD = Continuous Delivery / Deployment → Automatically deliver or deploy the application after successful testing.

CI vs CD in simple words

Term	Meaning	Example
CI	Integrate code continuously	Run tests after every GitHub push
CD (Delivery)	Prepare app for release	Create executable or package
CD (Deployment)	Automatically publish	Upload website to server

Remember: CI checks quality. CD delivers the product.

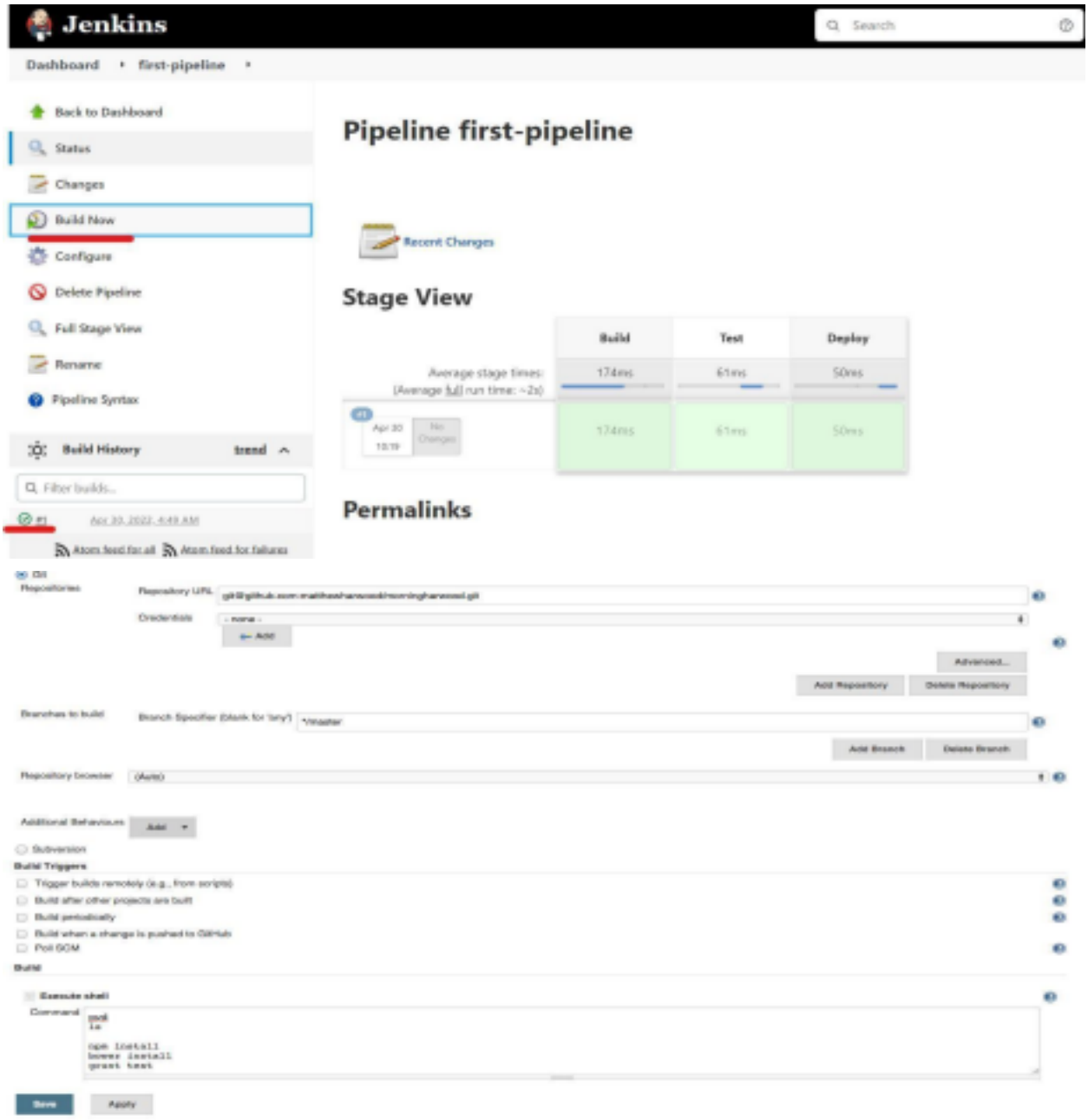
Complete CI/CD Pipeline Flow



Advantages of CI/CD

- Reduces manual work through automation.
- Detects errors early.
- Runs tests consistently after code changes.
- Saves time during software development.
- Improves software quality and reliability.

What is Jenkins?



5

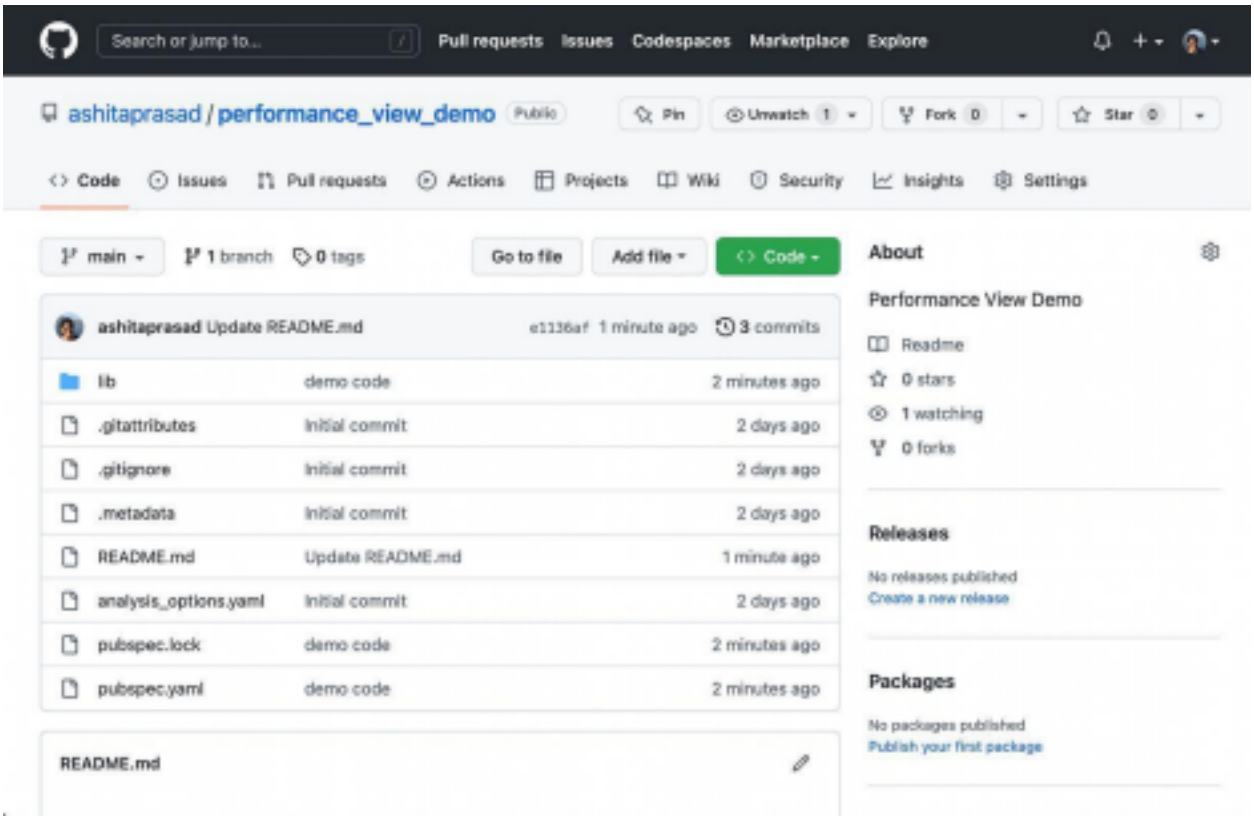
Jenkins is an automation server.

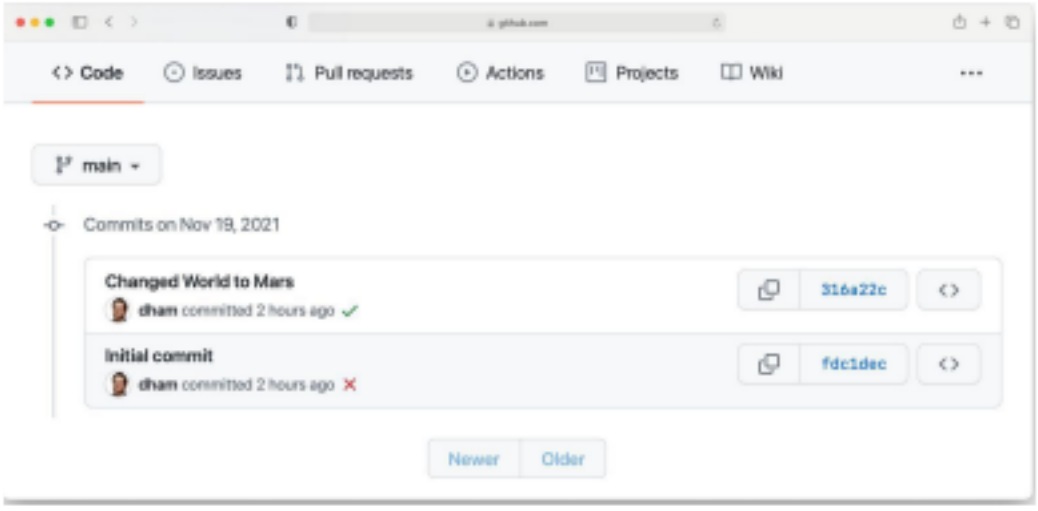
Think of it as a robot assistant that watches your GitHub repository.

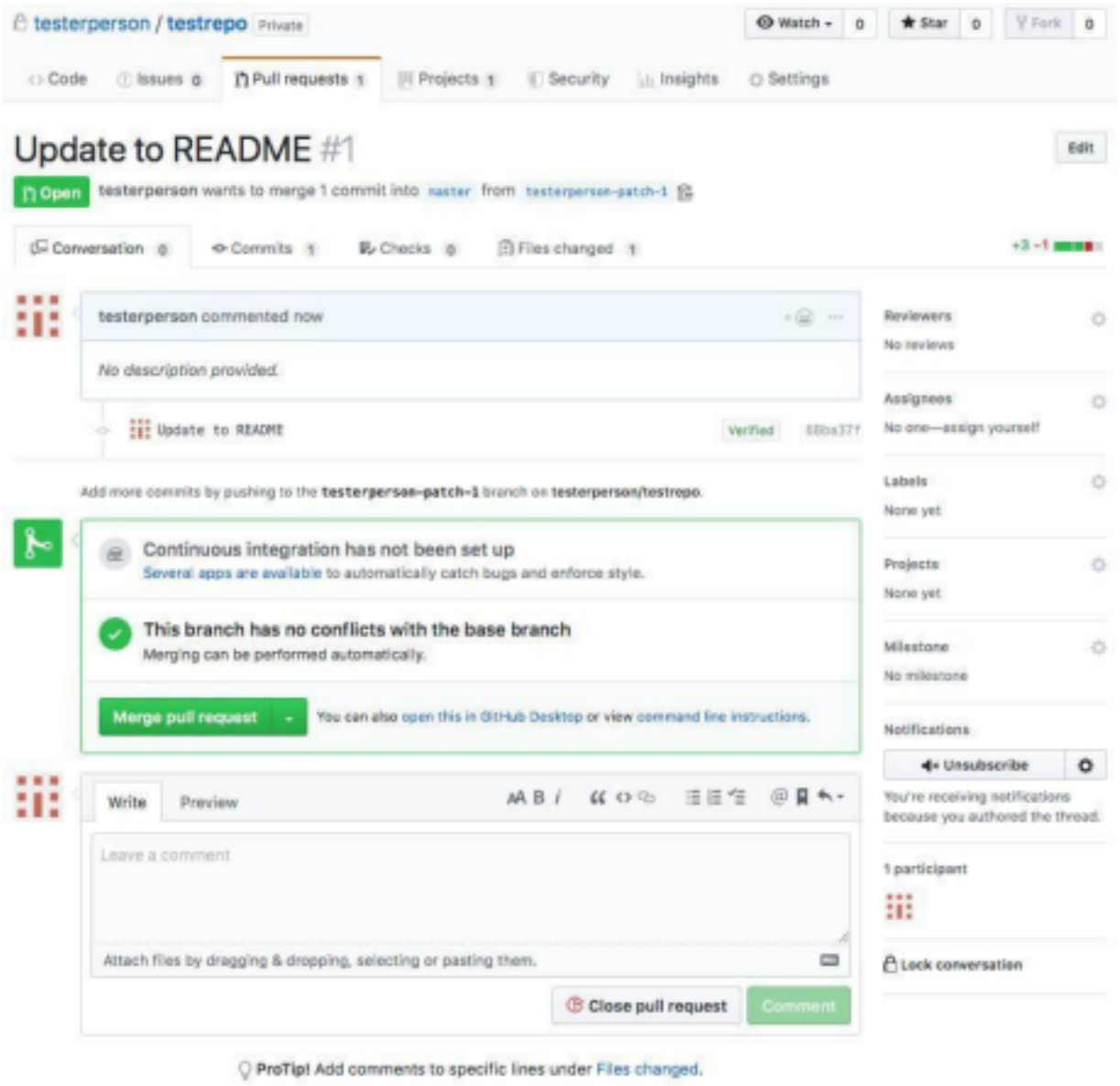
Whenever new code arrives, Jenkins automatically:

- Downloads the latest code
- Compiles it (if needed)
- Runs tests
- Generates reports
- Deploys the project

What is GitHub?







5

GitHub stores your source code online.

Example:

```
AI_Project/
  ├── app.py
  ├── test.py
  ├── requirements.txt
  └── README.md
```

Every time you click Commit & Push, GitHub receives the new version.

Then what is a Webhook?

A webhook is simply a notification.

Instead of Jenkins asking GitHub every minute:

"Any new code? Any new code?"

GitHub immediately tells Jenkins:

"Hey Jenkins! New commit received."



Webhook = GitHub automatically informs Jenkins.

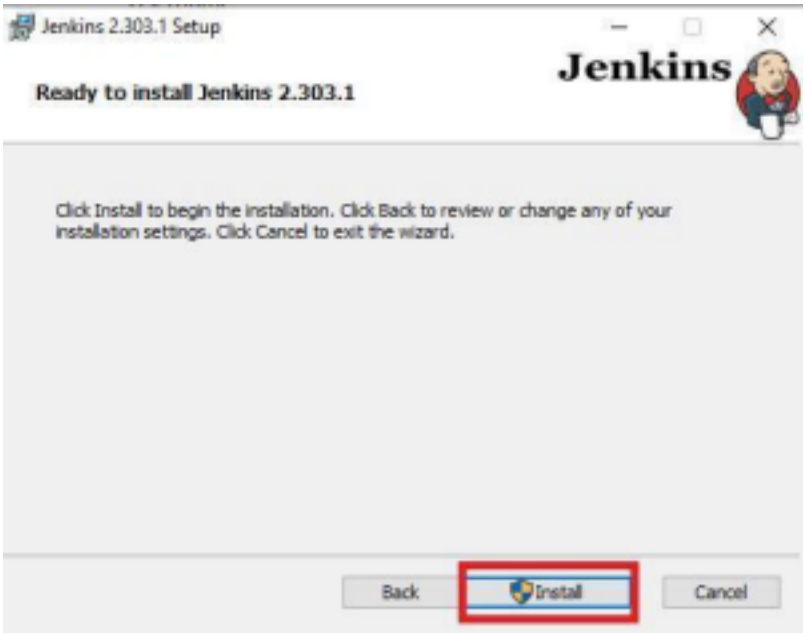
Complete Architecture



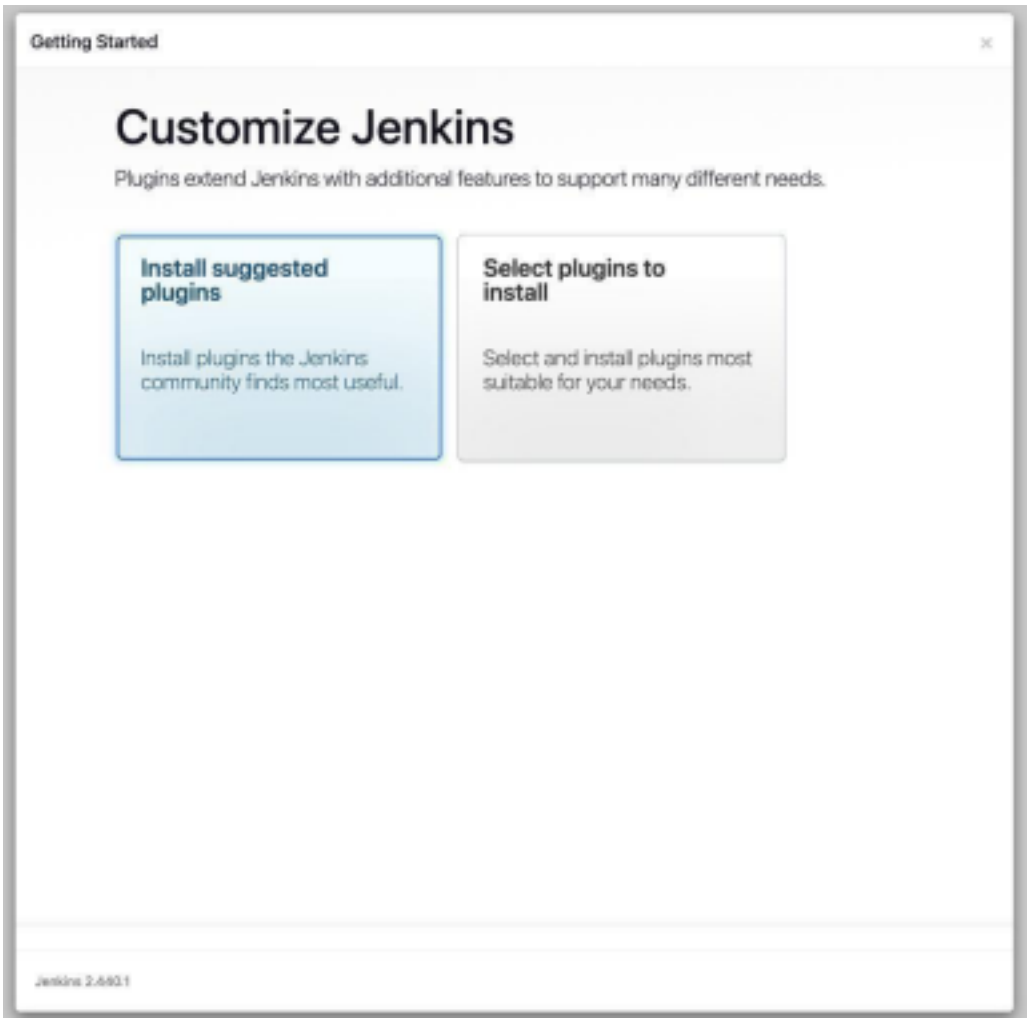
Software you'll need

Software	Purpose
Git	Version control
GitHub Account	Store code
Java JDK	Required for Jenkins
Jenkins	CI/CD automation
VS Code	Write code

Step 1: Install Jenkins


Jenkins 2.303.1 Setup
Ready to install Jenkins 2.303.1
Jenkins
Click Install to begin the installation. Click Back to review or change any of your installation settings. Click Cancel to exit the wizard.
Back
Install
Cancel


Getting Started
Unlock Jenkins
To ensure Jenkins is securely set up by the administrator, a password has been written to the log (not sure where to find it?) and this file on the server:
/var/jenkins_home/secrets/initialAdminPassword
Please copy the password from either location and paste it below.
Administrator password



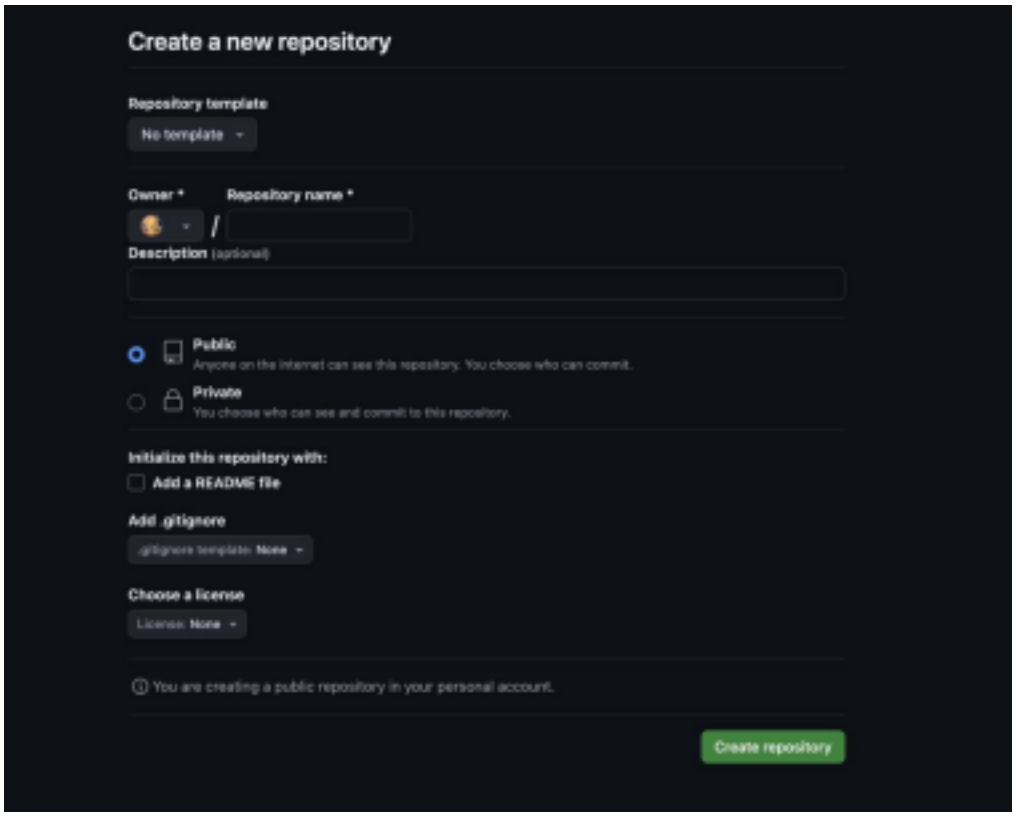
4

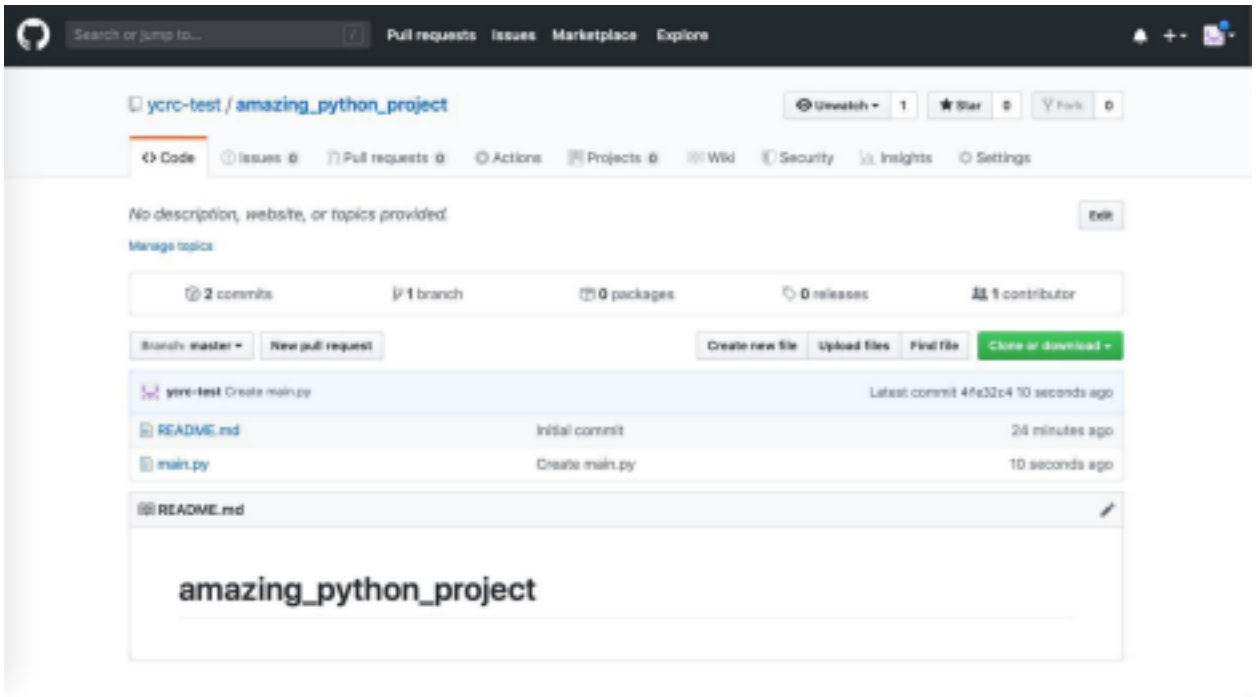
After installing:

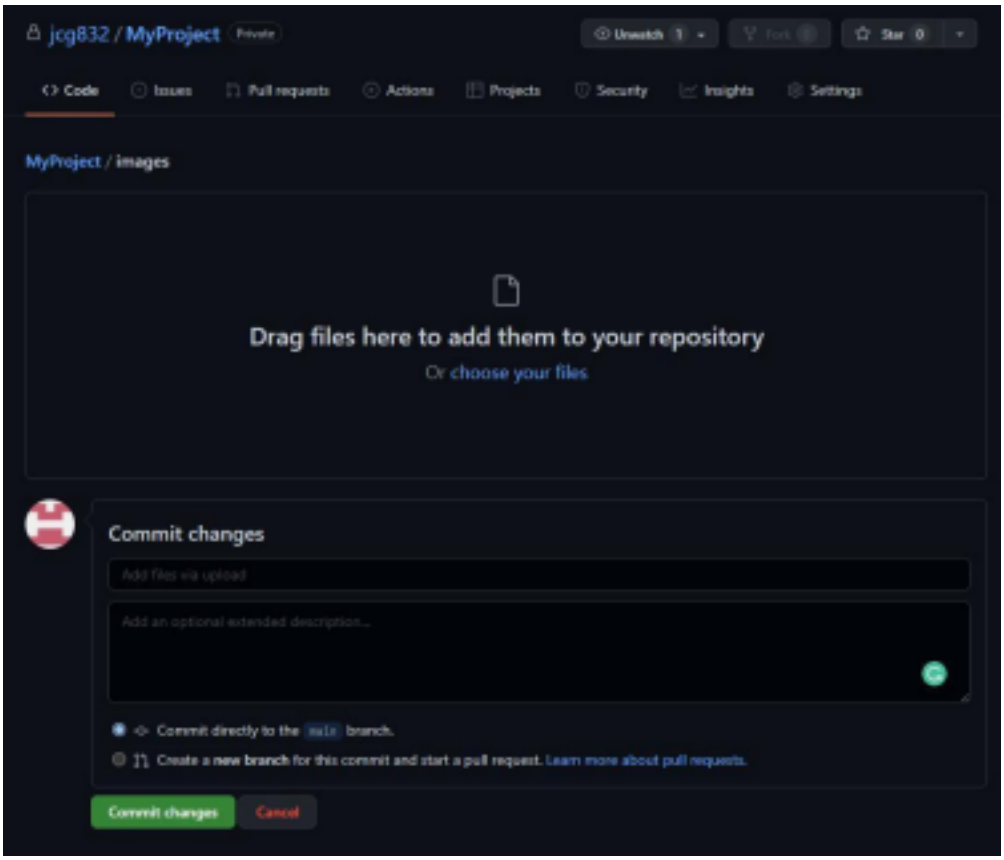
1. Open browser
2. Visit http://localhost:8080
3. Unlock Jenkins using the password file
4. Install suggested plugins
5. Create admin user

Your dashboard will look similar to the images above.

Step 2: Create GitHub Repository


Create a new repository
Repository template
No template
Owner *
Repository name *
/
Description (optional)
Public
Anyone on the internet can see this repository. You choose who can commit.
Private
You choose who can see and commit to this repository.
Initialize this repository with:
Add a README file
Add .gitignore
.gitignore template: None
Choose a license
License: None
You are creating a public repository in your personal account.
Create repository


Search or jump to...
Pull requests
Issues
Marketplace
Explore
ycrc-test / amazing_python_project
Unwatch 1
Star 0
Fork 0
Code
Issues 0
Pull requests 0
Actions
Projects 0
Wiki
Security
Insights
Settings
No description, website, or topics provided.
Edit
Manage topics
2 commits
1 branch
0 packages
0 releases
1 contributor
Branch: master
New pull request
Create new file
Upload files
Find file
Clone or download
ycrc-test Create main.py
Latest commit 4fe32c4 10 seconds ago
README.md
Initial commit
24 minutes ago
main.py
Create main.py
10 seconds ago
README.md
amazing_python_project



5

Create a repository named:

Jenkins-Demo

Add these files.

app.py

print("Hello CI/CD Pipeline")

test.py

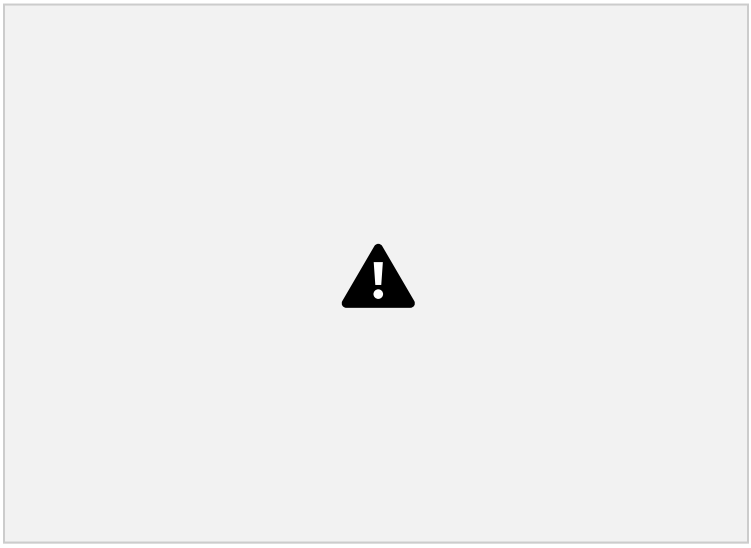
print("Testing successful")

Commit and push.

Step 3: Create Jenkins Job

In Jenkins:

New Item → Pipeline → Name: Python-CI



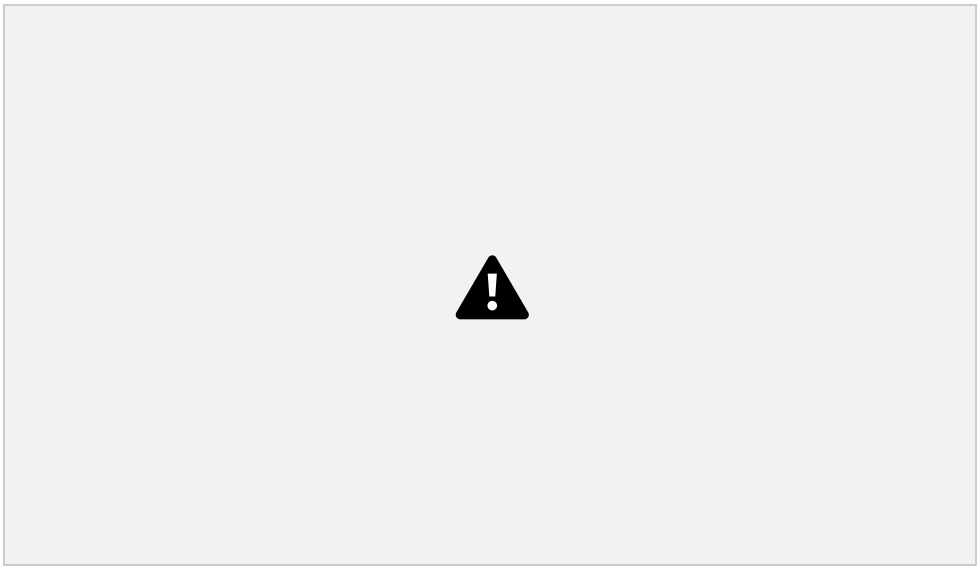
Inside configuration:

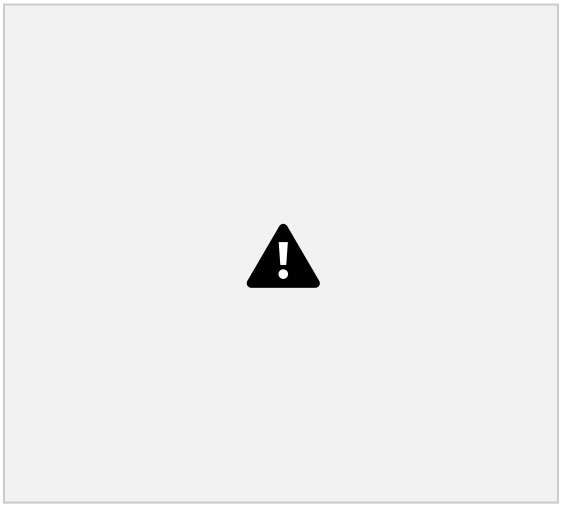
- Select Pipeline
- Save

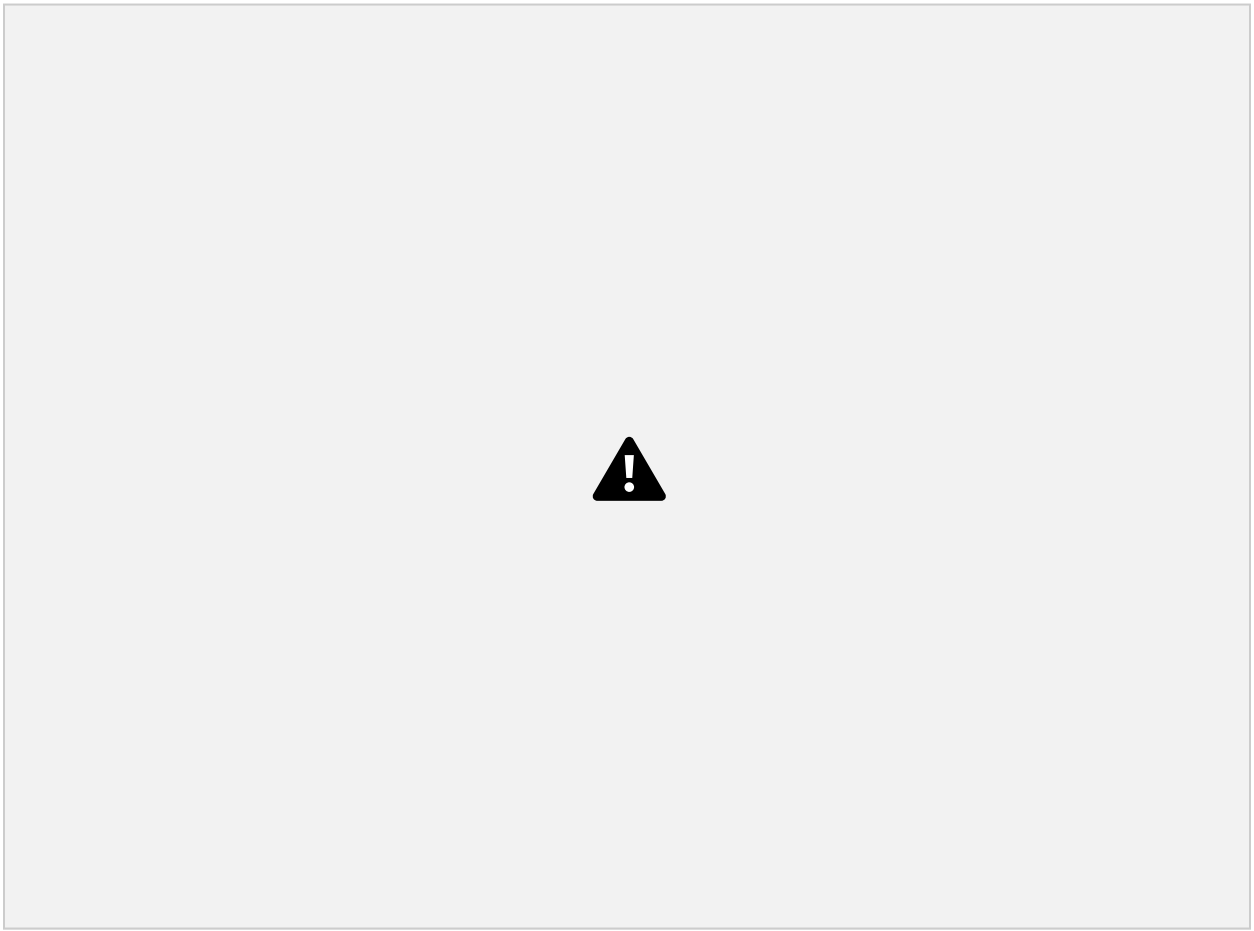
This job will execute our automation.

Step 4: Connect Jenkins with GitHub

Inside Jenkins job configuration:







6

Enable:

- GitHub Project
- GitHub Hook Trigger for GITScm Polling

Now copy the Jenkins webhook URL.

Example:

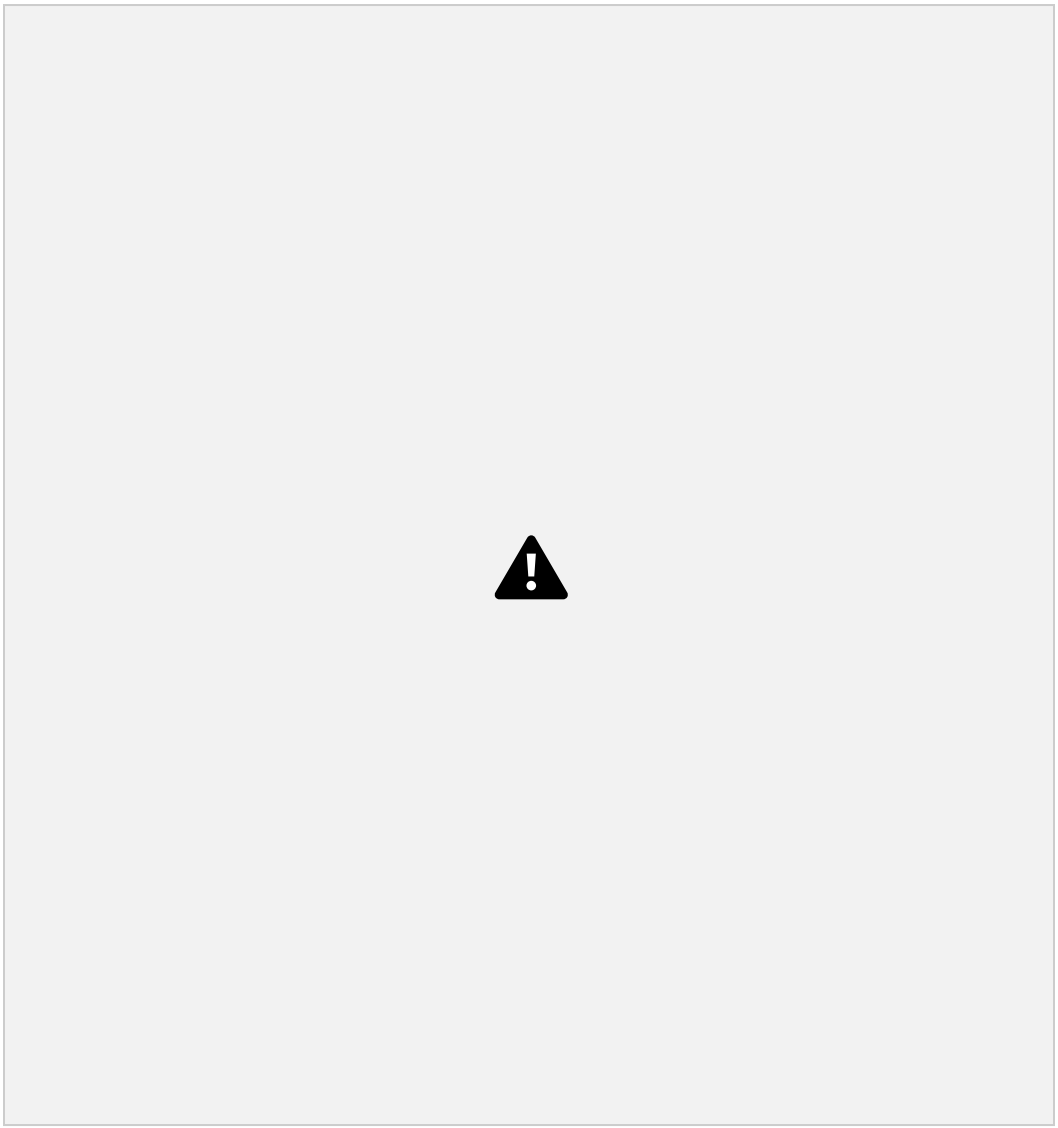
http://YOUR-IP:8080/github-webhook/

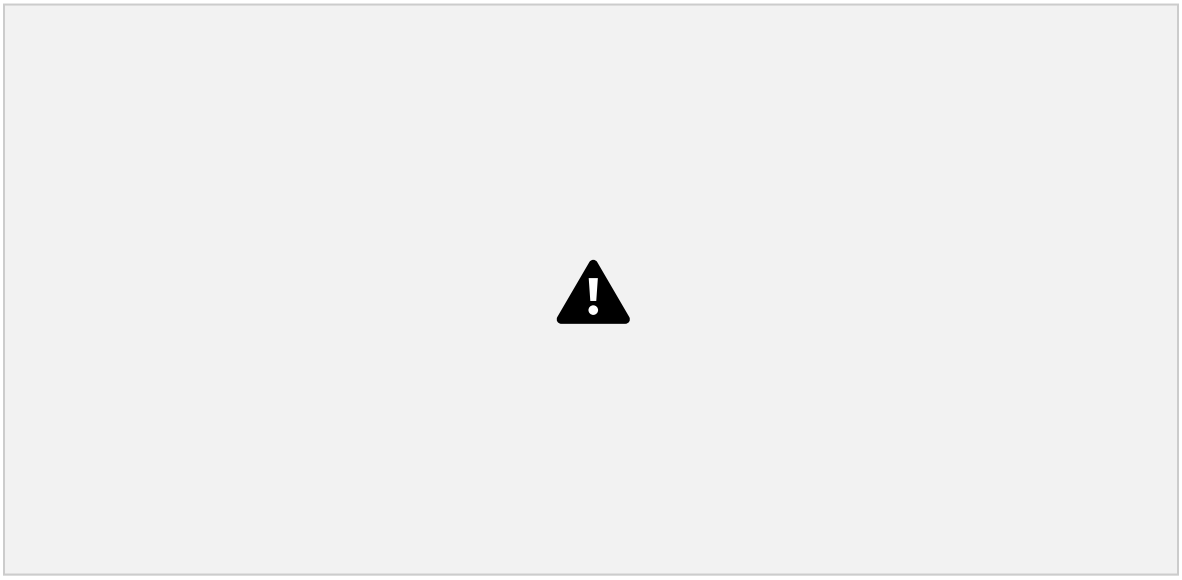
For local practice, many students use Ngrok so GitHub can reach localhost.

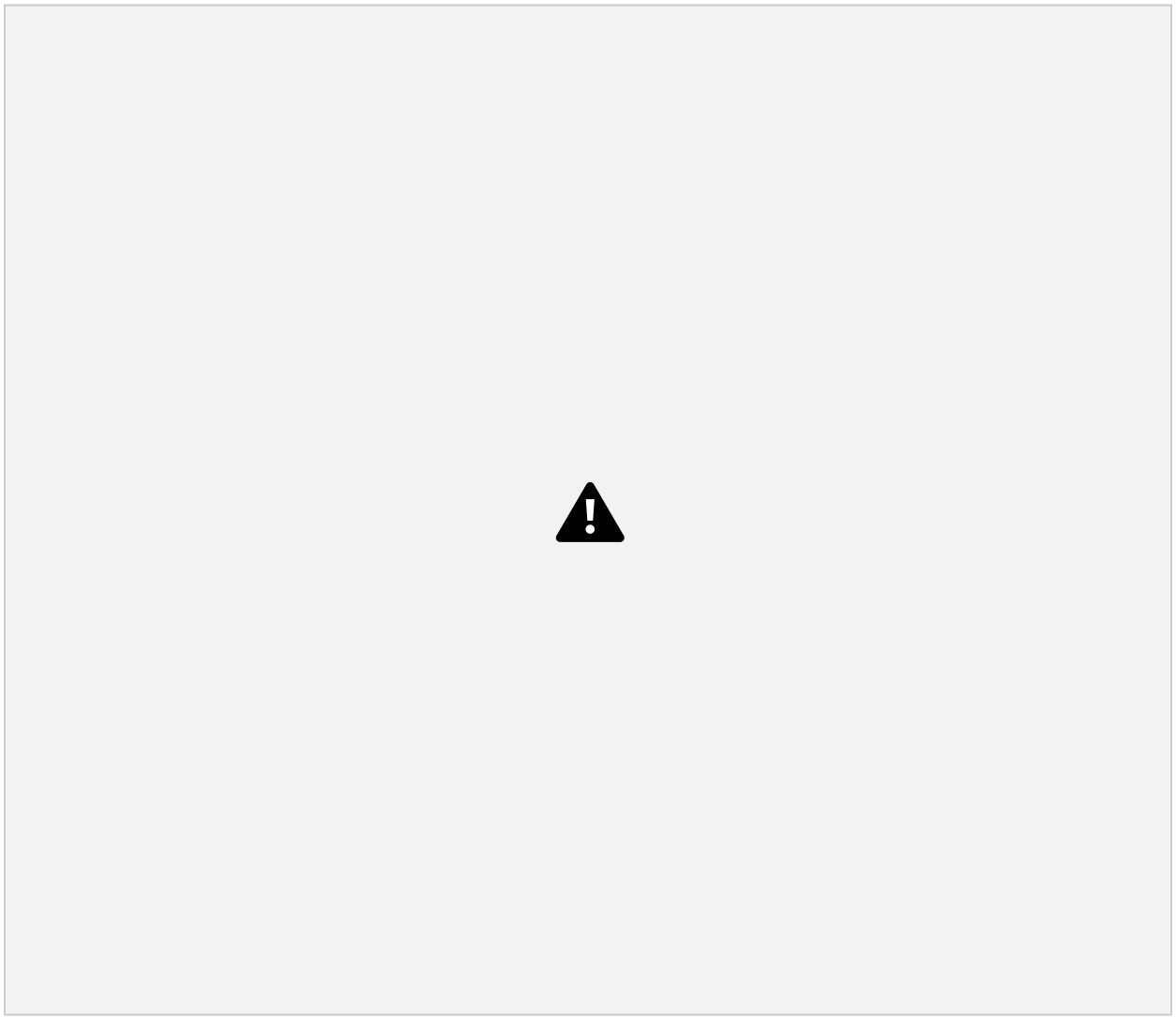
Step 5: Add GitHub Webhook

GitHub Repository:

Settings → Webhooks → Add Webhook







4
Fill:

Field	Value
Payload URL	Jenkins webhook URL

Content Type	application/json
Events	Just the push event

Click Add webhook.

Now GitHub and Jenkins are connected.

Step 6: Write Jenkins Pipeline

This file is called Jenkinsfile.

```
pipeline {
    agent any

    stages {

        stage('Clone') {
            steps {
                git 'https://github.com/USERNAME/Jenkins-Demo.git'
            }
        }

        stage('Build') {
            steps {
                bat 'python app.py'
            }
        }

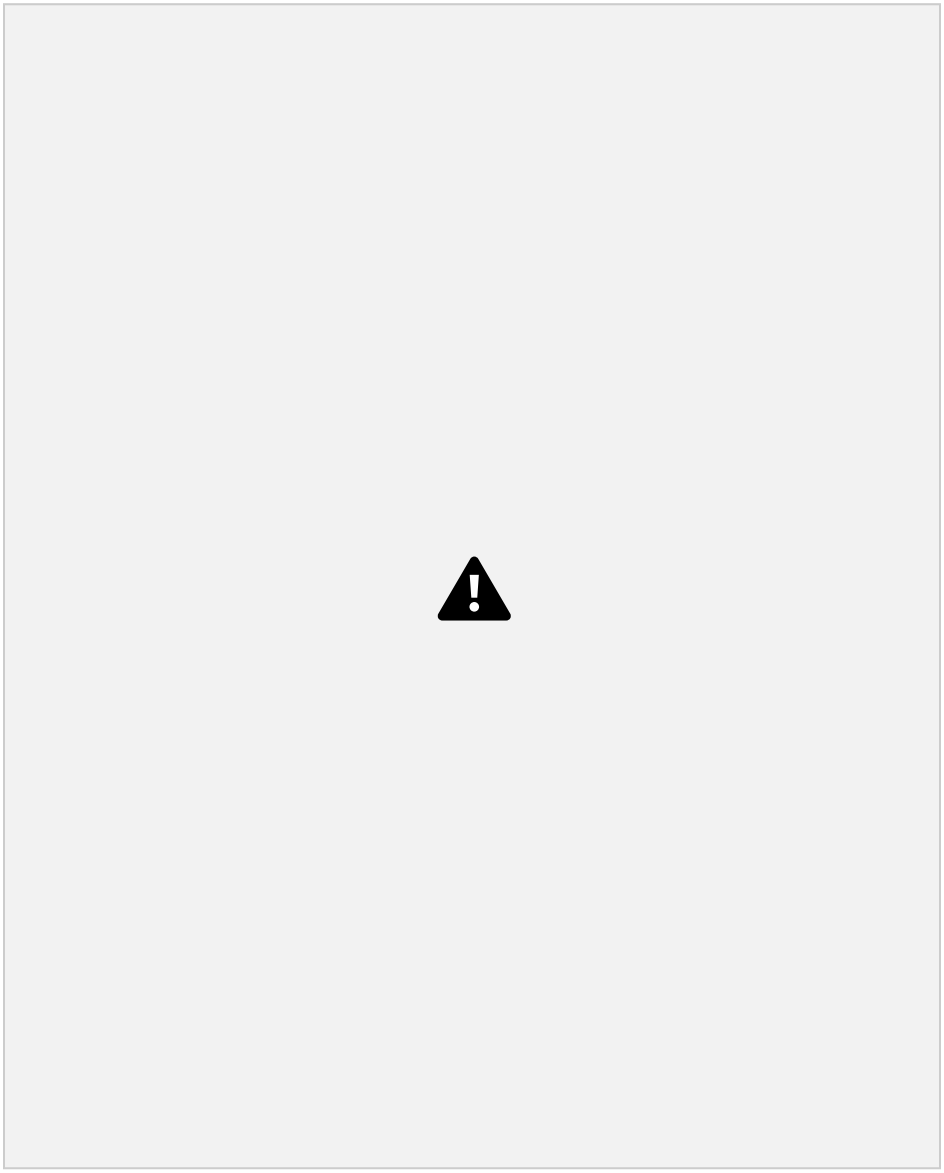
        stage('Test') {
            steps {
                bat 'python test.py'
            }
        }
    }
}
```

What each stage means

Stage	Work
Clone	Download code from GitHub
Build	Run application
Test	Execute test file

Jenkins runs them one by one.

What happens after Push?



Suppose you change:

print("Version 2")

Then run:

```
git add .
git commit -m "Updated app"
git push
```

Immediately:

1. GitHub receives code
2. Webhook sends message
3. Jenkins starts automatically

4. Build runs
5. Test runs
6. Green success appears

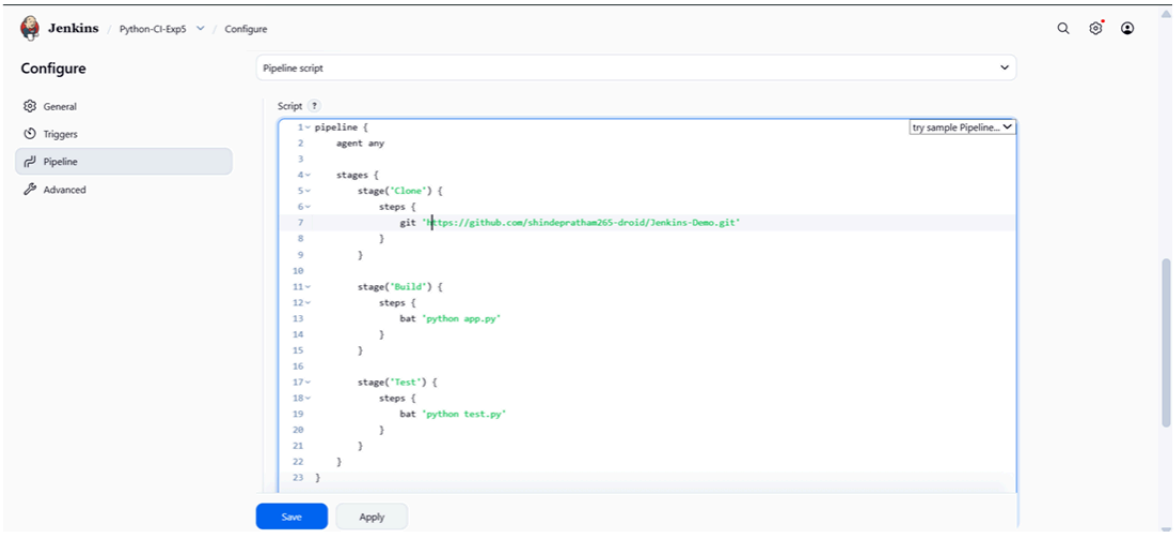
No manual clicking!

PART B
(PART B: TO BE COMPLETED BY STUDENTS)

Roll No. A21	Name: Pratham Dilip Shinde
Class: BE–AI&DS	Batch: A1
Date of Experiment: 01/09/2026	Date of Submission: 01/09/2026

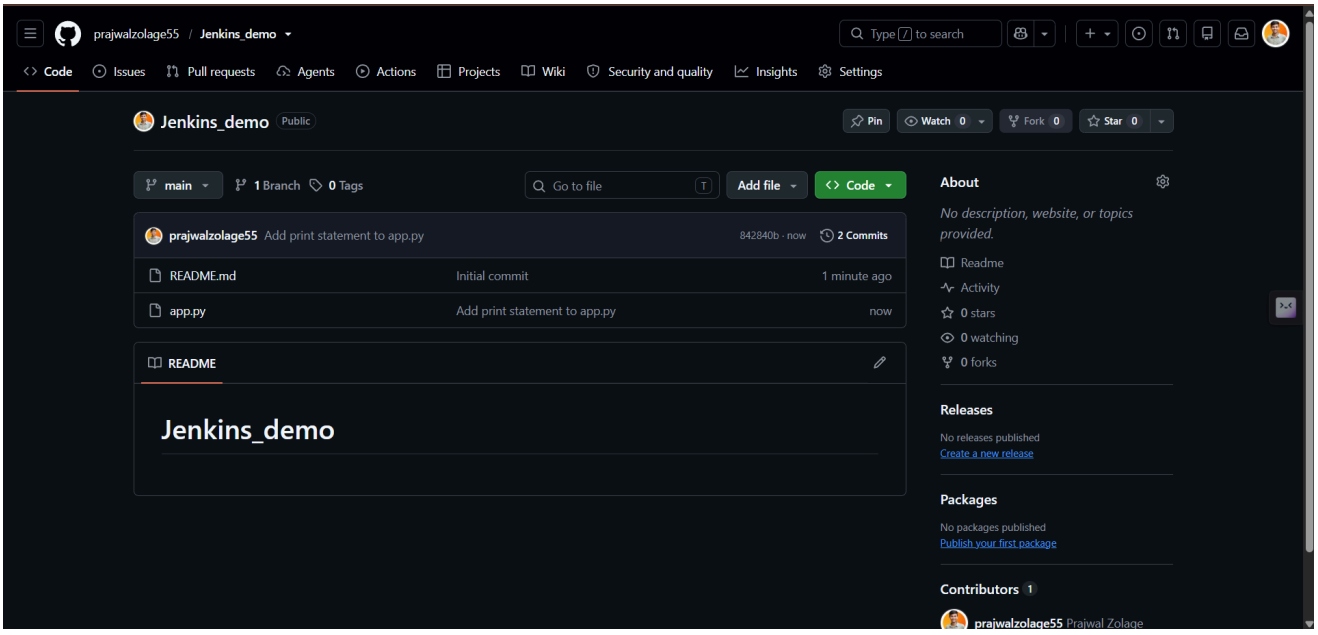
B.1 Code Screenshot:

- **Jenkins pipeline script**

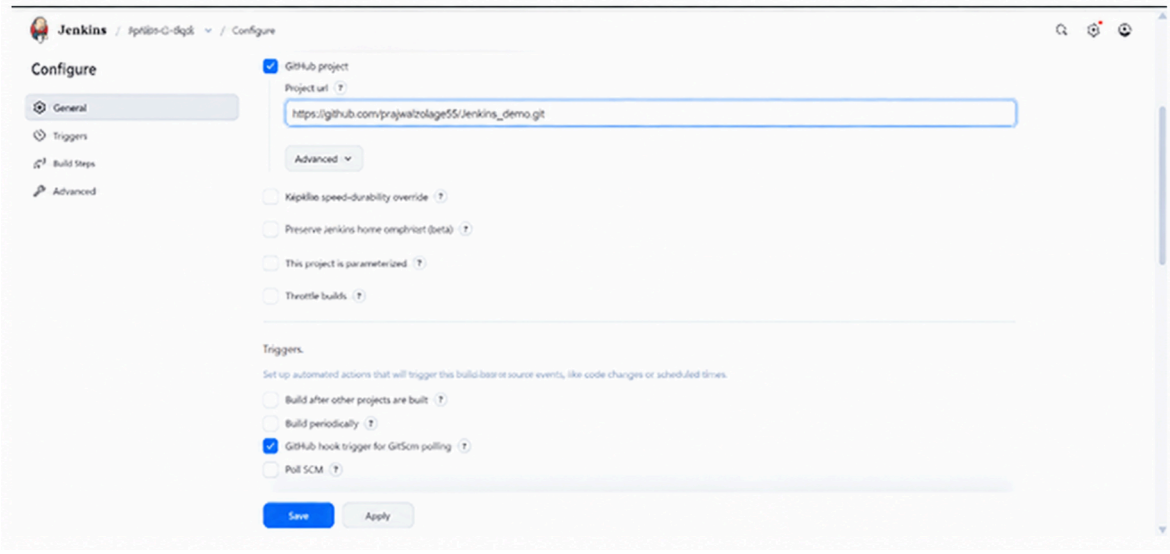


B.2 Input and Output:

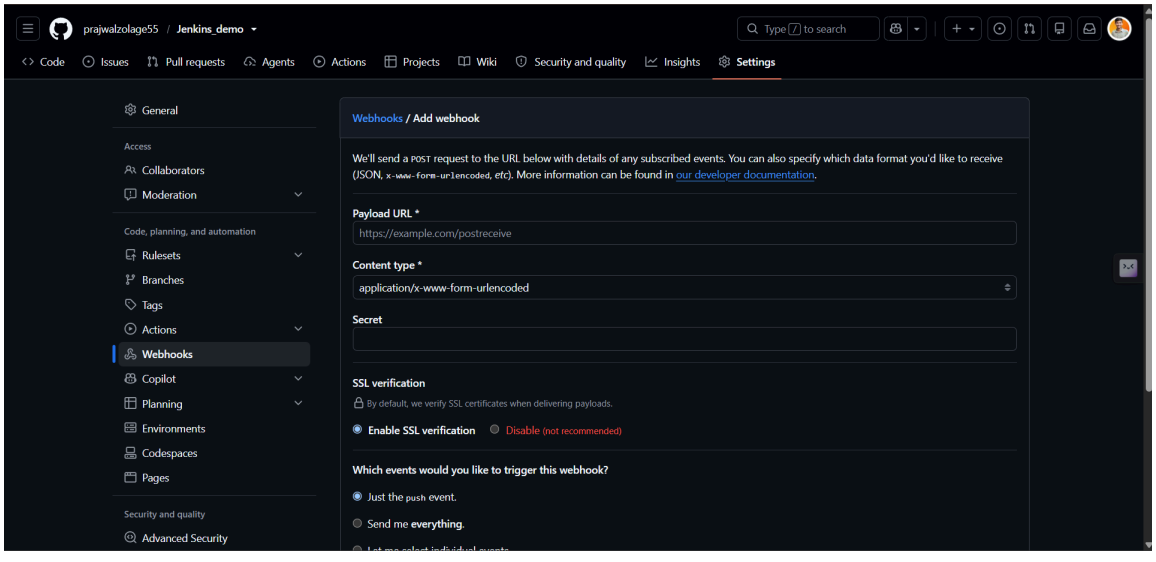
1. GitHub repository with files



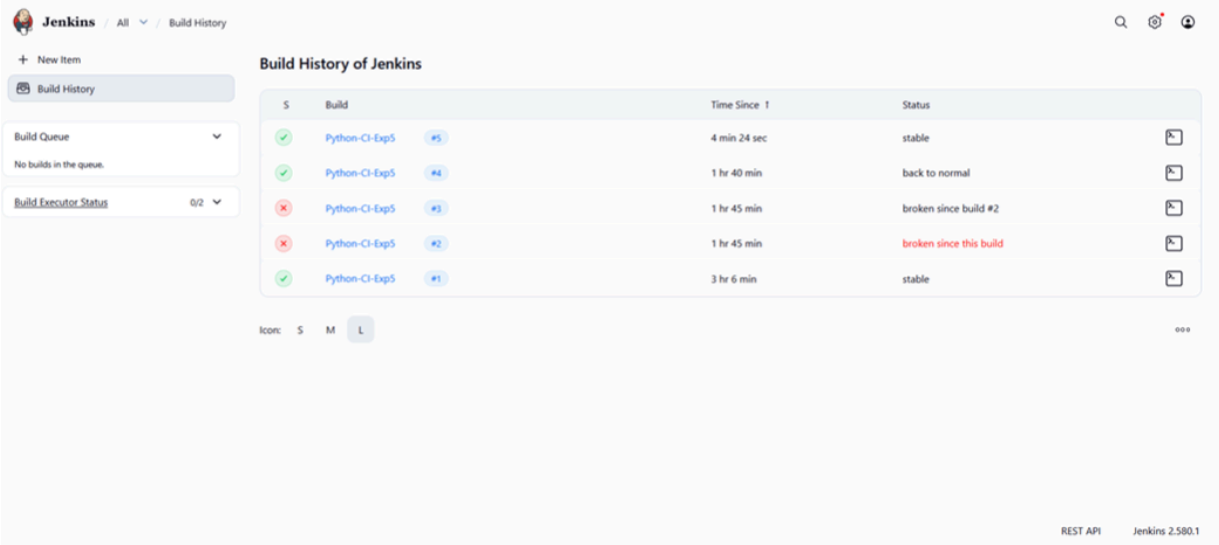
2. Jenkins GitHub project and hook trigger configuration



3. webhook last delivery successful

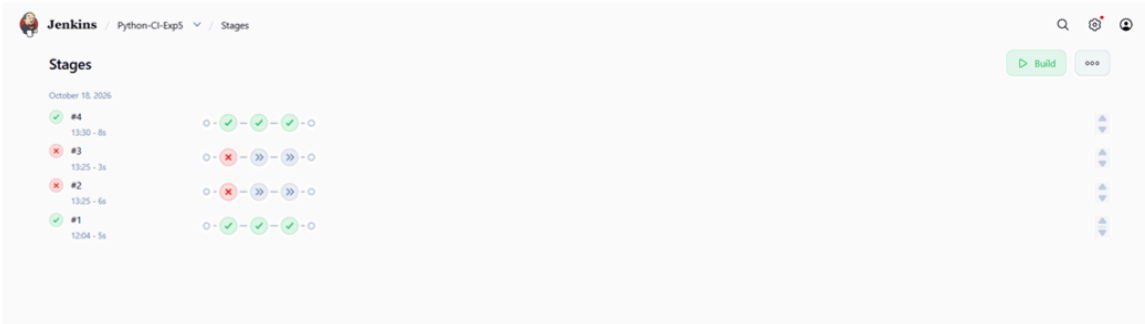


4. Build History (#1 to #5)



5. Stages view

6. Stages view



7. Console output of build #5, “Finished: SUCCESS”



B.3 Conclusion:

Thus, an automated CI/CD pipeline was successfully implemented using Jenkins and GitHub Webhooks. The pipeline automatically triggered when changes were committed to the GitHub repository. Jenkins cloned the source code, executed the Python application, and ran the test script. The successful build verified that the CI/CD pipeline was working correctly and reduced the need for manual execution.

****